

$$a^0 = 1 [a0]$$

$$\arcsin(z)$$

Dart Programming Fundamentals

Mukhammadali Khayotov

$$x_{n+1} =$$

Contents

- Main Function
- Variables
- Control Flow
- Functions
- Comments
- Classes
- Enums
- Inheritance
- Mixins
- Polymorphism
- Async
- Exceptions

Main Function

```
void main() {  
  print('Hello, World!');  
}
```

- An entrypoint to the app
- Returns void
- Can have parameters of type List<String>

P.S.: not to be confused with **runApp()** which attaches the given widget to the screen.

Main Function *(An example with List<String>)*

// Run the app like this: dart args.dart 1 test

```
void main(List<String> arguments) {  
  
    print(arguments);  
    assert(arguments.length == 2);  
    assert(int.parse(arguments[0]) == 1);  
    assert(arguments[1] == 'test');  
}
```

- An example provided above is given for the command-line app that takes some arguments.
- [args library](#) can be used to define or parse command-line arguments.

Variables

- Type Safe
 - Uses Static Type checking and Runtime Type checking to ensure that variable's value always matches it's static type. (Also referred as sound typing)
- Type Inference
 - Compiler can automatically determine the data type of a variable or expression based on its value.

Variables *(implicit declaration)*

```
var name = 'Voyager I';  
var year = 1977;  
var antennaDiameter = 3.7;  
var flybyObjects = ['Jupiter', 'Saturn', 'Uranus', 'Neptune'];  
var image = {  
  'tags': ['saturn'],  
  'url': '//path/to/saturn.jpg'  
};
```

- The data types are automatically set by a compiler (Type Inference)

Variables *(explicit declaration)*

```
String name = 'Voyager I';  
int year = 1977;  
double antennaDiameter = 3.7;
```

- The data types are explicitly set by a user

Variables *(Built-in Types)*

The Dart language has special support for the following:

- **Numbers** (`int`, `double`)
- **Strings** (`String`)
- **Booleans** (`bool`)
- **Records** (`(value1, value2)`)
- **Lists** (`List`, also known as *arrays*)
- **Sets** (`Set`)
- **Maps** (`Map`)
- **Runes** (`Runes`; often replaced by the `characters` API)
- **Symbols** (`Symbol`)
- The value `null` (`Null`)

Control Flow Statements *(Selection: If Statements)*

```
if (isRaining()) {  
    you.bringRainCoat();  
} else if (isSnowing()) {  
    you.wearJacket();  
} else {  
    car.putTopDown();  
}
```

- If else statements with else if

Control Flow Statements *(Selection: Switch)*

```
var command = 'OPEN';
switch (command) {
  case 'CLOSED':
    executeClosed();
  case 'PENDING':
    executePending();
  case 'APPROVED':
    executeApproved();
  case 'DENIED':
    executeDenied();
  case 'OPEN':
    executeOpen();
  default:
    executeUnknown();
}
```

- If else statements with else if

Control Flow Statements *(Selection: Switch)*

```
switch (command) {  
  case 'OPEN':  
    executeOpen();  
    continue newCase; // Continues executing at the newCase label.  
  case 'DENIED': // Empty case falls through.  
  case 'CLOSED':  
    executeClosed(); // Runs for both DENIED and CLOSED,  
  newCase:  
  case 'PENDING':  
    executeNowClosed(); // Runs for both OPEN and PENDING. }  
}
```

Control Flow Statements *(Iteration: For Loops)*

```
var message = StringBuffer('Dart is fun');  
for (var i = 0; i < 5; i++) {  
    message.write('!');  
}
```

For Iterable types such as List or Set...

```
for (final candidate in candidates) {  
    candidate.interview();  
}
```

For Iterable types you can use `forEach()` as well

```
var collection = [1, 2, 3];  
collection.forEach(print); // 1 2 3
```

Control Flow Statements *(Iteration: While Loops)*

```
while (!isDone()) {  
    doSomething();  
}
```

Do while loop will execute at least one time unlike the conventional while as it first does and then checks every iteration.

```
do {  
    printLine();  
} while (!atEndOfPage());
```

Control Flow Statements *(Iteration: break and continue)*

Break is used to stop the loop.

```
while (true) {  
    if (shutdownRequested()) break;  
    processIncomingRequests();  
}
```

Continue is used to skip to the next iteration in the loop

```
for (int i = 0; i < candidates.length; i++) {  
    var candidate = candidates[i];  
    if (candidate.yearsExperience < 5) {  
        continue;  
    }  
    candidate.interview();  
}
```

Control Flow Statements *(assert)*

If the condition in assert is true, then the code continues to execute, otherwise, it returns an assertion error and the program is stopped.

```
void main() {  
    String geek = "Geeks For Geeks";  
    assert(geek != "Geeks For Geeks");  
    print("You can see this line as output");  
}
```

Assert is usually useful, when debugging to check for presence of any logical errors

P.S.: just like many other programming languages Dart also uses ternary operations with a similar syntax to Java and C++

Functions

```
int fibonacci(int n) {  
    if (n == 0 || n == 1)  
        return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

```
var result = fibonacci(20);
```

Quick question: what kind of function is it in the above code?

- 1.No arguments and no return type:**
These functions do not take any arguments and do not return any value.
- 2.With arguments and no return type:**
These functions take one or more arguments but do not return any value.
- 3.No arguments and return type:** These functions do not take any arguments but return a value.
- 4.With arguments and with return type:**
These functions take one or more arguments and return a value.

Functions

A shorthand arrow ($\Rightarrow$) operator can also be used for single statement functions. This is recommended when Passing anonymous functions as arguments.

```
flybyObjects.where((name) => name.contains('turn')).forEach(print);
```

The code on the right is a mere example to understand how where() and forEach() work together Where() and forEach() are usually used with collections

```
void main() {  
    List<int> numbers = [1, 2, 3, 4, 5];  
    var evenNumbers = numbers.where((number) => number % 2 == 0);  
    print(evenNumbers); // Output: (2, 4)  
}
```

P.S:

Functions in Dart are first-class objects, which means they can be assigned to variables or passed as arguments to other functions

You can also call an instance of a Dart class as if it were a function

Functions

- Dart also has closures (which may also be referred as function objects)
- Because it is declared within a scope it will have access to the statements within that scope (like local variables)

```
void main() {  
  int x = 10;  
  
  Function addX = () {  
    x++;  
    print(x);  
  };  
  
  addX(); // Output: 11  
  addX(); // Output: 12  
}
```

Comments

`// This is a normal, one-line comment.`

`/// This is a documentation comment, used to document libraries,
/// classes, and their members. Tools like IDEs and dartdoc treat /// doc comments
specially.`

`/* Multi-line comments are also supported. */`

- Comments in dart are very common in many other programming languages such as C++ and Java.

Comments

```
/// A domesticated South American camelid (Lama glama).  
///  
/// Andean cultures have used llamas as meat and pack  
/// animals since pre-Hispanic times.  
///  
/// Just like any other animal, llamas need to eat,  
/// so don't forget to [feed] them some [Food].
```

```
class Llama {  
    String? name;  
  
    /// Feeds your llama [food].  
    ///  
    /// The typical llama eats one bale of hay per week.  
    void feed(Food food) {  
        // ...  
    }  
    /// Exercises your llama with an [activity] for  
    /// [timeLimit] minutes.  
    void exercise(Activity activity, int timeLimit) {  
        // ...  
    }  
}
```

```
void main() {  
    /*  
    * This is a lot of work. Consider raising chickens.  
  
    Llama larry = Llama();  
    larry.feed();  
    larry.exercise();  
    larry.clean();  
    */  
}
```

Imports

// Importing core libraries

```
import 'dart:math';
```

// Importing libraries from external packages

```
import 'package:test/test.dart';
```

// Importing files

```
import 'path/to/my_other_file.dart';
```

- To access APIs defined in other libraries (similar to import in Java or #include in C/C++)

Imports *(show, hide, deferred, as)*

```
import 'package:lib1/lib1.dart';  
import 'package:lib2/lib2.dart' as lib2;
```

```
// Uses Element from lib1.  
Element element1 = Element();
```

```
// Uses Element from lib2.  
lib2.Element element2 = lib2.Element();
```

```
// Import only foo.  
import 'package:lib1/lib1.dart' show foo;
```

```
// Import all names EXCEPT foo.  
import 'package:lib2/lib2.dart' hide foo;
```

```
import 'package:greetings/hello.dart' deferred as hello;
```

//library is not loaded until it is invoked by //loadLibrary() identifier.

```
Future<void> greet() async {  
  await hello.loadLibrary();  
  hello.printGreeting();  
}
```

Class *(hierarchy)*

Dart is an object-oriented language with classes and mixin-based inheritance. Every object is an instance of a class, and all classes except Null descend from Object.

In Dart, a class can have only one superclass, but a class body can be reused in multiple class hierarchies using mixins.

Mixins are a way to reuse a class's code in multiple class hierarchies. They are similar to interfaces in other languages, but they can also provide implementation details.

```
Object
├── Comparable
│   ├── DateTime
│   └── Duration
├── Iterable
│   ├── List
│   │   ├── Queue
│   │   └── Set
│   └── Map
│       └── Set
├── MapEntry
├── Pattern
│   ├── RegExp
│   └── String
├── Sink
│   └── StringSink
├── Stream
│   ├── BroadcastStream
│   ├── ByteStream
│   ├── StreamController
│   ├── StreamIterator
│   └── StreamView
├── StreamSubscription
├── StringBuffer
├── StringSinkMixin
└── Symbol
```

Classes

```
class Spacecraft {
    String name;
    DateTime? launchDate;

    // Read-only non-final property
    int? get launchYear => launchDate?.year;

    // Constructor, with syntactic sugar for assignment to members.
    Spacecraft(this.name, this.launchDate) {
        // Initialization code goes here.
    }

    // Named constructor that forwards to the default one.
    Spacecraft.unlaunched(String name) : this(name, null);

    // Method.
    void describe() {
        print('Spacecraft: $name');
        // Type promotion doesn't work on getters.
        var launchDate = this.launchDate;
        if (launchDate != null) {
            int years = DateTime.now().difference(launchDate).inDays ~/ 365;
            print('Launched: $launchYear ($years years ago)');
        } else {
            print('Unlaunched');
        }
    }
}
```

What is syntactic sugar?

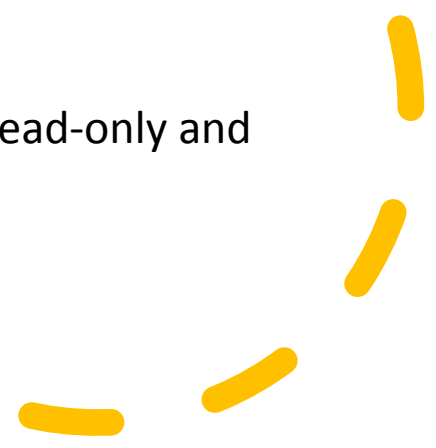



```
var voyager = Spacecraft('Voyager I', DateTime(1977, 9, 5));  
voyager.describe();
```

```
var voyager3 = Spacecraft.unlaunched('Voyager III');  
voyager3.describe();
```

Classes

•**Quick question:** What is the difference between read-only and final property?



Enums

Enums can look like this:

```
enum PlanetType { terrestrial, gas, ice }
```

Enums are a way of enumerating a predefined set of values or instances in a way which ensures that there cannot be any other instances of that type

P.S.: Do not confuse/compare Enums in Swift and Dart. In Dart, Enums are specially designed to represent some number of constants while in Swift, there are a lot of functionalities associated with Enums.

Enums

Or... like this:

```
/// Enum that enumerates the different planets in our solar system
/// and some of their properties.
enum Planet {
  mercury(planetType: PlanetType.terrestrial, moons: 0, hasRings: false),
  venus(planetType: PlanetType.terrestrial, moons: 0, hasRings: false),
  // ...
  uranus(planetType: PlanetType.ice, moons: 27, hasRings: true),
  neptune(planetType: PlanetType.ice, moons: 14, hasRings: true);

  /// A constant generating constructor
  const Planet(
    {required this.planetType, required this.moons, required this.hasRings});

  /// All instance variables are final
  final PlanetType planetType;
  final int moons;
  final bool hasRings;

  /// Enhanced enums support getters and other methods
  bool get isGiant =>
    planetType == PlanetType.gas || planetType == PlanetType.ice;
}
```

Enums

- Usage of the above enum class

```
final yourPlanet = Planet.earth;

if (!yourPlanet.isGiant) {
    print('Your planet is not a "giant planet".');
}
```

Inheritance

Quick question: what is a diamond problem in inheritance? When does it occur?

```
class Orbiter extends Spacecraft {  
    double altitude;  
  
    Orbiter(super.name, DateTime super.launchDate, this.altitude);  
}
```

- Unlike many other programming languages such as C++ or Java, dart uses Single Inheritance.



Mixins

```
mixin Piloted {  
    int astronauts = 1;  
  
    void describeCrew() {  
        print('Number of astronauts: $astronauts');  
    }  
}
```

```
class PilotedCraft extends Spacecraft with Piloted {  
    // ...  
}
```

The good thing about mixin is that you can use multiple of them (i.e.: extends Spacecraft with Mixin1, Mixin2 ...{}

The major difference between multiple mixins against to ordinary multiple inheritance is that Mixins do not make the class which is inheriting a parent.

Interfaces and Abstract Classes (Polymorphism)

```
class MockSpaceship implements Spacecraft {  
    // ...  
}
```

This is very similar to other programming languages;

```
abstract class Describable {  
    void describe();  
  
    void describeWithEmphasis() {  
        print('=====');  
        describe();  
        print('=====');  
    }  
}
```

Abstract class is being created to be later implemented by
A concrete class

Async

```
const oneSecond = Duration(seconds: 1);  
// ...  
Future<void> printWithDelay(String message) async {  
    await Future.delayed(oneSecond);  
    print(message);  
}
```


Async

```
Future<void> printWithDelay(String message) {  
    return Future.delayed(oneSecond).then((_ ) {  
        print(message);  
    });  
}
```

Async

- **Async** is a keyword in Dart that is used to mark a function as asynchronous. When a function is marked as asynchronous, it means that it can perform long-running operations without blocking the **main thread**. The function returns a Future object that can be used to retrieve the result of the operation when it completes¹.
- On the other hand, a **callback** is a function that is passed as an argument to another function. The callback function is called when the operation completes. In Dart, callbacks can be synchronous or asynchronous².
- In summary, **async** is used to mark a function as asynchronous, while **callbacks** are used to handle the result of an asynchronous operation.

Async

```
Future<void> createDescriptions(Iterable<String> objects) async {  
  for (final object in objects) {  
    try {  
      var file = File('$object.txt');  
      if (await file.exists()) {  
        var modified = await file.lastModified();  
        print(  
          'File for $object already exists. It was modified on $modified.');
```


 continue;
 }
 await file.create();
 await file.writeAsString('Start describing \$object in this file.');
 } on IOException catch (e) {
 print('Cannot create description for \$object: \$e');
 }
 }
}

Exceptions

```
if (astronauts == 0) {  
    throw StateError('No astronauts.');
```

To raise an exception, we need to use throw keyword.

```
Future<void> describeFlybyObjects(List<String> flybyObjects) async {  
    try {  
        for (final object in flybyObjects) {  
            var description = await File('$object.txt').readAsString();  
            print(description);  
        }  
    } on IOException catch (e) {  
        print('Could not describe object: $e');  
    } finally {  
        flybyObjects.clear();  
    }  
}
```

To catch an exception, we need
To use try statement with on or
Catch or both.

Exceptions

```
try {  
  // Code that might throw an exception.  
} catch (e) {  
  // Code to handle the exception.  
} finally {  
  // Code that will always be executed.  
}
```

In Dart, the **try** statement is used to enclose a block of code that might throw an exception.

The **catch** statement is used to handle the exception if it is thrown.

The **finally** statement is used to specify a block of code that will be executed regardless of whether or not an exception is thrown¹.

The **finally** block is optional, but if it is present, the code inside it will always be executed, even if an exception is thrown. This can be useful for cleaning up resources or releasing locks, for example